Úa: Using the compiled assembly

uv, uvh and h

Hilmar Gudmundsson

31 October 2026

Contents

1 Overview **1**

2 Quick start on Windows **2**

3 Recommended settings **3**

4 What is supported, and the automatic fallback **4**

5 Checking that it is used **5**

6 Building the MEX files yourself on Windows **6**

7 When to rebuild **7**

8 Parallelisation, threads and shared machines **8**
8.1 How the compiled assembly is parallelised 8
8.2 PARDISO 10
8.3 No parallel pool needed 11
8.4 Setting the number of threads 11

9 Linux **12**

10 Troubleshooting **13**

11 Regression test **14**

12 Performance **14**

1 Overview

Úa can assemble its three main nonlinear systems with compiled code instead of vectorised MATLAB code: the uv system (velocities), the uvh system (velocities and thickness, implicit time step) and the h system (thickness, with the velocities given). The results are the same to rounding errors, and the solvers take the same time steps and Newton iterations. Each assembly has its own switch:

Switch	System	Assembly, and where it is used
`CtrlVar.uvMatrixAssembly.CompiledKernel`	uv	`uvMatrixAssemblySSTREAM.m`: every uv solve, ie diagnostic runs, the start of transient runs, the uv-h scheme, and inverse runs (forward solves and adjoint)
`CtrlVar.uvhMatrixAssembly.CompiledKernel`	uvh	`uvhMatrixAssembly.m`: the implicit uvh time steps
`CtrlVar.hMatrixAssembly.CompiledKernel`	h	`MassContinuityEquationAssembly.m`: the h solves of the uv-h scheme, and `MassContinuityEquation.m`

The compiled code is generated from Úa's own MATLAB functions with MATLAB Coder, and runs in parallel over the elements on all processor cores (OpenMP). It is a drop-in replacement: no other option, input file or physics changes.

- **Default:** all three switches are off (`false`), so nothing changes unless you switch them on.
- **Fallback:** if an option, element type or the MEX files are not available, or the MEX files are outdated or fail, Úa uses the vectorised assembly instead, and says why once per session.
- **No parallel pool needed:** the compiled assembly is parallel by itself. The spmd options for the uv and uvh assembly are no longer useful with it (Section 3).
- **Gain:** a single assembly becomes 6 to 18 times faster; whole runs 1.5 to 4 times faster (Section 12). A Greenland run with the uv-h scheme, for example, went from 314 to 108 s.
- **Tested:** a regression test (`runtests("TestUaCompiled")`, 88 tests, about 30 s) compares the compiled with the vectorised assembly for all options (Section 11).

The compiled code lives in `UaSource/UaCompiled`. Ready-built MEX files for Windows (64-bit) are included; on Linux you build them once yourself (Section 9).

How the compiled assembly works, and what to observe when changing the code it is built from, is described in the separate document *Úa: Using the compiled assembly – Notes for Developers*.

2 Quick start on Windows

On 64-bit Windows with MATLAB R2026b, the included MEX files work as they are: you need neither MATLAB Coder nor a C compiler.

1. Put Úa on the path as usual. `UaSource/UaCompiled` is included by `genpath`:

```
addpath(genpath('C:\...\UaSource'))
```

2. In `DefineInitialInputs.m`, add:

```
CtrlVar.uvMatrixAssembly.CompiledKernel=true;
CtrlVar.uvhMatrixAssembly.CompiledKernel=true;
CtrlVar.hMatrixAssembly.CompiledKernel=true;
```

```
CtrlVar.Parallel.uvAssembly.spmd.isOn=false;    % the compiled assembly is
    parallel already
CtrlVar.Parallel.uvhAssembly.spmd.isOn=false;
```

3. Run Úa as usual. Nothing else changes, and no message appears while the compiled assembly is used.

You can also switch it on from a run script, without editing `DefineInitialInputs.m`:

```
CtrlVarOnInput.uvMatrixAssembly.CompiledKernel=true;
CtrlVarOnInput.uvhMatrixAssembly.CompiledKernel=true;
CtrlVarOnInput.hMatrixAssembly.CompiledKernel=true;
CtrlVarOnInput.Parallel.uvAssembly.spmd.isOn=false;
CtrlVarOnInput.Parallel.uvhAssembly.spmd.isOn=false;
Ua(UserVar,CtrlVarOnInput)
```

The switches are independent: you can, for example, switch on only the uv assembly. The included MEX files were built with MATLAB R2026b. MEX files generated by MATLAB Coder generally only run in the release they were built with, so with another release, build them yourself (Section 6).

3 Recommended settings

Use all three compiled assemblies, without spmd, and PARDISO as the linear solver if you can.

Setting	Recommended	Why
`CtrlVar.uvMatrixAssembly.CompiledKernel`, `CtrlVar.uvhMatrixAssembly.CompiledKernel`, `CtrlVar.hMatrixAssembly.CompiledKernel`	`true`	Switch the compiled uv, uvh and h assembly on (default `false`).
`CtrlVar.Parallel.uvAssembly.spmd.isOn`, `CtrlVar.Parallel.uvhAssembly.spmd.isOn`	`false`	With spmd on, these assemblies run on the spmd workers, and the compiled kernel is not used there at all. The compiled assembly is parallel already, and much faster: with spmd (4 workers), one uv assembly on a Greenland mesh (111,000 elements) took 360 ms, against 589 ms serial, and only about half of that was the workers' assembly work; the rest was overhead of the spmd approach (Section 8.1). No parallel pool is needed.

Setting	Recommended	Why
`CtrlVar.LinearSolver.Type`	`"PARDISO"`, if the Intel MKL runtime is available	Not part of the compiled assembly, but once the assembly is fast, the linear solves take a large share of the time. See `Ua2D_DefaultParameters.m` for the setup. The uv, uvh and h solvers pass the cached assembly pattern of the mesh to PARDISO, so that its analysis is done once per mesh.

For a Greenland run with the uv-h scheme (nx150, 111,000 elements, 20 time steps), this combination took 108 s, against 314 s with the vectorised assemblies and spmd for the uv assembly.

4 What is supported, and the automatic fallback

The compiled assemblies support all options of the corresponding vectorised assemblies, chosen at run time (no rebuild needed when they change), with linear, quadratic and cubic elements and quadrature rules of up to 37 points. Where they cannot be used, Úa falls back to the vectorised assembly and prints the reason once per MATLAB session.

uv assembly: all sliding laws, with all their names and aliases (see `SlidingLawCode.m`); the mélange terms; `ZeroFields`; full and residual-only assembly. The outputs `Ruv`, `Tint` and `Fext` are full vectors, and `Kuv` is symmetrised, as in the vectorised assembly.

uvh assembly:

- all numeric parameters in `CtrlVar`, including the time step; all sliding laws; the mélange terms;
- all SUPG tau options: `"tau1"`, `"tau2"`, `"taus"`, `"taut"`, `"tau0"`;
- the thickness penalty (softplus, polynomial, exponential), the thickness barrier, `ResetThicknessToMinThickness`, `Enforce_bAboveB`;
- level-set and mass-balance geometry feedback; `OnlyCalcBasalDragAndEffectiveViscosity` (used by `CalcBasalTraction.m`).

The diagnostic plots and output inside the uvh assembly (`doplots` with `PlotSUPGparameter`, and `InfoLevelThickMin>=10`) are not produced by the compiled code. To see them, switch the compiled uvh assembly off for that run.

h assembly: level-set and mass-balance geometry feedback; the thickness penalty at the integration points and nodal (`ThicknessPenaltyNodal`); nodes with thickness constraints; all SUPG tau options. If the matrix `dFdt` is requested (fourth output, for stability analysis), the vectorised assembly is used, without a message. The mass balance is always evaluated at the nodes: evaluation at integration points (`CtrlVar.MassBalance.Evaluation="-int-"`) is no longer available, and gives an error at the start of a run.

Complex-valued input (uv and h assembly) is an error, not a fallback: the kernels are compiled for real values.

Situation	What happens	What to do
`CtrlVar.Parallel.uvAssembly.spmd.isOn=true` or `CtrlVar.Parallel.uvhAssembly.spmd.isOn=true`	That assembly runs vectorised on the spmd workers. No message.	Set it to `false`.
`UaSource/UaCompiled` not on the path	Vectorised assembly; message: *. . . UaSource/UaCompiled is not on the path. . .*	Add it, eg with `addpath(genpath(UaSource))`.
MEX files not found (another operating system, not built yet)	Vectorised assembly; the message names the missing MEX files.	Build them (Sections 6 and 9).
MEX files outdated: a source file changed since they were built, or built with another MATLAB release	Vectorised assembly; the message names the changed files or the release.	Rebuild them (Section 6).
A MEX file fails when called (eg *Invalid MEX-file*, or inputs that no longer match)	Vectorised assembly for the rest of the MATLAB session; the message, printed once, gives the error.	Rebuild them; then `clear CompiledAssemblyCheck`, or restart MATLAB.
Elements with more than 10 nodes, or quadrature rules with more than 37 points	Vectorised assembly; the message names the element type.	None needed; rare.
Mesh without the cached assembly pattern (`MUA.uvAssemblyPattern`, `MUA.uvhAssemblyPattern`, `MUA.hAssemblyPattern`)	Vectorised assembly; the message names the pattern.	Keep the defaults `CtrlVar.MUA.AssemblyPattern.uv`, `.uvh` and `.h` (all `true`). Meshes read from older files get the patterns from `UpdateMUA.m`.
Unknown sliding law or tau option, or a `CtrlVar` field missing (eg an old restart file)	Vectorised assembly; the message names the options.	Correct the option, or check `CtrlVar` against `Ua2D_DefaultParameters.m`.

The messages start with the name of the assembly: *uvMatrixAssembly:*, *uvhMatrixAssembly:* or *hMatrixAssembly:*.

5 Checking that it is used

With the switches on and spmd off, no message means that the compiled assembly is used. To confirm it explicitly:

- **After (part of) a run,** list the loaded kernel MEX files. Their names start with `uvKernel`, `uvhKernel` and `hKernel`; for linear elements with 6 integration points they end in `_n3i6_mex`:

```
[~,m]=inmem; m(contains(m,"Kernel"))
```

- **With the profiler,** `uvMatrixAssemblyCompiled`, `uvhMatrixAssemblyCompiled`, `hMatrixAssemblyCompiled` and the kernel MEX files appear in the profile.
- **Before a long run,** for a given `CtrlVar` and mesh, and each assembly (`"uv"`, `"uvh"`, `"h"`):

  ```
  [ok,~,KernelFull]=CompiledAssemblyCheck(CtrlVar,MUA,"uv")
  ```

  `ok=true` means the compiled assembly will be used, with the MEX file named in `KernelFull`; otherwise the reason is printed.

If you want to see for yourself that the results are the same, run a few time steps with the switches off and on and compare: the solutions agree to rounding errors (in our tests, ice thickness to about 10^{-11} m), with the same numbers of time steps and Newton iterations. The regression test does this systematically (Section 11).

6 Building the MEX files yourself on Windows

You only need to build the MEX files if you use a MATLAB release other than R2026b, or after changes to the code they are built from (Section 7). Building all of them takes about 7 minutes and needs MATLAB Coder and a C compiler with OpenMP.

Requirements

- MATLAB Coder (a separate licence).
- A C compiler that MATLAB supports, with OpenMP. Tested: MinGW-w64 14.2, installed through the Add-On Explorer as *MATLAB Support for MinGW-w64 C/C++/Fortran Compiler*. Microsoft Visual C++ should also work (the build script switches on OpenMP for it), but has not been tested.

Steps

1. Select the C compiler once:

   ```
   mex -setup C
   ```

2. Build in a **fresh** MATLAB session. A MEX file that has been called in a session stays loaded and cannot be replaced (also not from another MATLAB session that has it loaded), so build before running Úa, or from the command line:

   ```
   matlab -batch "addpath(genpath('C:\...\UaSource')); BuildUaCompiled"
   ```

3. `BuildUaCompiled` writes 13 MEX files to `UaSource/UaCompiled`, and the build information `UaCompiledBuildInfo_mexw64.mat` (the MATLAB release and checksums of the source files of each MEX file, see Section 7); the intermediate files go to the temporary folder.
4. Check the build with the regression test (Section 11):

   ```
   results=runtests("TestUaCompiled"); table(results)
   ```

Variants	MEX files	Used for
`uvFull`, `uvRonly`, `uvFull_n3i6`, `uvRonly_n3i6`	`uvKernelFull_mex`, `uvKernelRonly_mex`, and the same with `_n3i6`	the uv assembly
`Full`, `Ronly`, `Full_n3i6`, `Ronly_n3i6`	`uvhKernelFull_mex`, `uvhKernelRonly_mex`, and the same with `_n3i6`	the uvh assembly
`hFull`, `hRonly`, `hFull_n3i6`, `hRonly_n3i6`	`hKernelFull_mex`, `hKernelRonly_mex`, and the same with `_n3i6`	the h assembly
`Sparse`	`SparseFromCSC`	creating the sparse matrices directly (hand-written C, shared)

`Full` computes the residual and the matrix, `Ronly` the residual only (used in the line searches). The `_n3i6` variants are for linear elements with 6 integration points (fixed sizes, the fastest), the others for all element types (up to 10 nodes and 37 integration points). To build only some of them, list them, for example the h assembly for linear elements only:

```
BuildUaCompiled(["hFull_n3i6","hRonly_n3i6"])
```

The resulting MEX files depend only on MATLAB's own libraries and the Windows runtime, not on the compiler, so they can be copied to other Windows computers with the same MATLAB release. Copy `UaCompiledBuildInfo_mexw64.mat` with them: without it, Úa cannot check whether they are up to date, and says so.

7 When to rebuild

The MEX files contain a compiled copy of the MATLAB functions listed below. Rebuild them whenever one of these changes, or when you change MATLAB release; options and parameters never need a rebuild.

Rebuild after:

- **a change of MATLAB release** (MEX files generated by MATLAB Coder generally only run in the release they were built with);
- **an update of Úa** (eg `git pull`) that changes any of these files:
  - in `UaCompiled`: the kernels `uvElementKernel.m`, `uvhElementKernel.m`, `hElementKernel.m`, their entry points (`uvKernelFull.m`, `uvKernelRonly.m`, `uvhKernelFull.m`, `uvhKernelRonly.m`, `hKernelFull.m`, `hKernelRonly.m`), `MakeKernelParamsFlex.m` and `SparseFromCSC.c`;
  - in `UaMain`: `BasalDrag.m`, `SlidingLawBeta.m`, `SlidingLawCode.m`, `EffectiveViscositySSTREAM.m`, `SUPGtau.m`, `SUPGtauCode.m`, `ThicknessPenaltyMassBalanceFeedback.m`, `LevelSetMethodMassBalanceFeedback.m`, `HeavisideApprox.m`, `DiracDelta.m`, `SmoothFloor.m`, `SoftPlus.m`, `shape_fun.m`.

No rebuild is needed for changes to options or parameters (sliding law, tau option, penalty

function, time step and so on), or to the other MATLAB files in `UaCompiled` (`uvMatrixAssemblyCompiled.m`, `uvhMatrixAssemblyCompiled.m`, `hMatrixAssemblyCompiled.m`, `CompiledAssemblyCheck.m`, `CompiledAssemblyPattern.m`).

Úa detects outdated MEX files itself. `BuildUaCompiled` records, for each MEX file, the MATLAB release and a checksum of each of its source files in `UaCompiledBuildInfo_<mexext>.mat`; once per MATLAB session and assembly, before the first compiled assembly, `CompiledAssemblyCheck` compares them (about 0.1 s). If a source file has changed since the build, or another MATLAB release is running, the vectorised assembly is used, with a message such as

```
uvMatrixAssembly: compiled kernel not used, as the MEX files are outdated: UaMain/
    SoftPlus.m changed since they
were built (rebuild them with BuildUaCompiled.m). The vectorised assembly is used
    instead.
```

Changed line endings alone (eg from a git checkout) do not count as a change. Without build information, a note says that this could not be checked, and the MEX files are used.

8 Parallelisation, threads and shared machines

The compiled assembly and PARDISO both run in parallel threads within the MATLAB process; neither needs a parallel pool. By default they use all processor cores; limit this with environment variables set before MATLAB starts (Section 8.4).

8.1 How the compiled assembly is parallelised

The kernels (`uvElementKernel.m`, `uvhElementKernel.m`, `hElementKernel.m`) contain `parfor` loops, but they are never run as MATLAB code: they are compiled into MEX files with MATLAB Coder, which translates each `parfor` loop into an OpenMP parallel loop in the generated C code (`#pragma omp parallel for`). At run time this means:

- **Threads in the MATLAB process itself:** no workers, no parallel pool, and no data transfer between processes. All threads work directly on the same arrays in memory (shared memory).
- **The number of threads** is by default one per logical processor (32 on a 16-core processor with hyperthreading). It is set with `OMP_NUM_THREADS`, not with MATLAB's `maxNumCompThreads` or a pool.

Each kernel call has three parallel loops:

1. **Over the elements**, the main work. Per element, a thread gathers the element's nodal values, computes all terms at all integration points (serially within the element), and writes the element residual and element matrix into *its own column* of element-wise arrays. No two iterations write to the same place, so there are no conflicts and no locks.
2. **Over the rows of the residual**: each thread adds up, for its rows, the contributions of all elements sharing that node, using lists of positions computed once per mesh.
3. **Over the nonzeros of the matrix**: the same for the matrix entries. For the uv assembly, a further parallel loop then symmetrises the matrix.

`SparseFromCSC` (hand-written C) then copies the finished arrays into the sparse matrix, also in parallel with OpenMP.

Why the summation is organised by target. The obvious approach would be to add each element's contributions directly into the shared residual and matrix from within the element loop. But neighbouring elements share nodes, so several threads would write to the same entry at the same time: that needs atomic operations or locks, and the order of the additions would vary from run to run, so the results would differ in the last digits. Instead, the element loop only writes into its own column, and each row and each nonzero is then computed by exactly one thread, in a fixed order (the same order as MATLAB's `accumarray`). The results are therefore reproducible from run to run, and bit-identical to the vectorised summation of the same element values.

What stays serial: the loops over integration points and nodes within an element, and the small MATLAB side of each assembly (preparing the input vectors, the check whether the compiled assembly can be used).

How well it scales (Greenland, 197,000 elements; best of several assemblies):

Threads	**uv assembly, full / residual only**	**uvh kernel, full**
1	1,065 / 524 ms	1,992 ms
4	361 / 172 ms	—
8	204 / 108 ms	394 ms
16	135 / 74 ms	295 ms
32 (default)	112 / 55 ms	269 ms

That is about 8 times faster with 16 cores than with one, and 9.5 times with hyperthreading. For comparison, the vectorised uv assembly takes 1,494 / 806 ms (serial): even with a single thread, the compiled assembly is faster. The element loop is pure arithmetic and scales almost perfectly; the two summation loops read scattered positions from memory, so memory bandwidth limits them more.

The difference from spmd. The spmd options of the vectorised uv and uvh assembly split the mesh into sub-domains, one per worker of a parallel pool. In every call (uv assembly, `uvMatrixAssemblySSTREAM_SPMD.m`):

1. the client sends the current velocities to the workers (other fields, `CtrlVar` and the boundary conditions only when they change);
2. each worker assembles its sub-domain with the vectorised assembly (the sub-domains have equal numbers of elements);
3. the partial results are summed with `spmdPlus`: each worker returns full-size residual vectors and a sparse matrix with the entries of its sub-domain; `spmdPlus` is a collective operation, so all workers have to take part, those that finish early wait for the slowest one, and the sparse matrices are added and passed between the workers;
4. the client fetches the summed results, including the whole sparse matrix, from worker 1.

Measured on a Greenland mesh (111,000 elements) with 4 workers (thread pool), median of 5 assemblies:

uv assembly	**Full (residual and matrix)**	**Residual only**
Serial vectorised, for comparison	589 ms	274 ms
Ideal with 4 workers (serial / 4)	147 ms	69 ms
spmd, measured	**360 ms**	**203 ms**
assembly on the 4 workers	178 / 182 / 202 / 205 ms	87 / 87 / 93 / 106 ms
waiting for the slowest worker	up to 27 ms	up to 19 ms
summing the sparse matrices (`spmdPlus`)	about 20 ms	—
starting, synchronising and ending the spmd block	about 49 ms	about 45 ms
fetching the results from worker 1 to the client	17 ms	11 ms
on the client (detecting changed fields, sending the velocities, comparing `CtrlVar`)	about 44 ms	about 25 ms

What this shows:

- **Waiting for the slowest worker is real, but not the largest overhead.** Although the four sub-domains have exactly the same number of elements, the workers differ by up to 15 % in their assembly time (scheduling, hyperthreading, memory access), and the summation can only start when the slowest worker delivers.
- **Starting, synchronising and ending the spmd block** cost about 45–49 ms per call, whatever the mesh size; fetching the results and the work on the client add another 36–61 ms. In a run with several hundred assemblies, this adds up considerably.
- **The workers themselves are slower than a quarter of the serial work** (178–205 instead of 147 ms): in a thread pool they share the memory bandwidth, and each runs single-threaded, whereas the serial assembly can use several threads for large vectorised operations.

With spmd, the uv assembly was therefore about 1.6 times faster than serial (residual only: 1.35 times), against 4 times ideally; about half of the time with spmd is the workers' assembly work, the other half overhead. The compiled assembly needs none of this: all threads work on the same arrays in one process, with no transfers, and the summation is organised so that no thread waits for another until the end of each loop.

A note on the profiler. On the client, MATLAB's profiler shows the whole spmd block as a single wait (`awaitBlock`): it includes the workers' actual assembly work as well as all overheads, as the profiler cannot see into the workers. A large waiting time in the profile is therefore not in itself idle time.

8.2 PARDISO

PARDISO parallelises itself as well. `UaPardisoSolve` loads Intel MKL (`mkl_rt`, through `pardisoMex`), and MKL computes the factorisation and the solve with its own threads, also within the MATLAB process. Its number of threads comes from MKL's own settings (`MKL_NUM_THREADS`, otherwise `OMP_NUM_THREADS`, otherwise the number of cores), not from a pool.

8.3 No parallel pool needed

With the compiled assembly and PARDISO, a parallel pool is not needed, and runs are better off without one:

Option or function that would use a pool	Default	With the compiled assembly
`CtrlVar.Parallel.uvAssembly.spmd.isOn`, `CtrlVar.Parallel.uvhAssembly.spmd.isOn`	`false`	keep `false`: with spmd, the vectorised assembly runs on the workers, and the compiled kernel is not used
`CtrlVar.Parallel.uvhAssembly.parfor`, `.hAssembly.parfor`, `.LSFAssembly.parfor`, `.uvAssembly.parfeval`	off	older parallel variants of the vectorised assemblies; not needed
`CalcBruteForceGradient.m`, `CalcBruteForceHessian.m`	on request	finite-difference checks of gradients and Hessians; these benefit from a pool

Without spmd, `UpdateMUA.m` also skips building the workers' mesh data (in a Greenland inversion: 2.2 s → 0.3 s). An open pool costs start-up time (a few seconds for a thread pool, more for a process pool) and, for a process pool, memory: each worker is a separate MATLAB process. Idle workers take no processor time, so they do not slow the compiled assembly or PARDISO down; but with a pool open, an spmd option left switched on takes effect, and silently switches the compiled kernel off for that assembly.

8.4 Setting the number of threads

- **Setting the number of threads.** The OpenMP runtime reads `OMP_NUM_THREADS` once, when the compiled code is first called, so set it before starting MATLAB, for example in a Windows command window:

  ```
  set OMP_NUM_THREADS=8
  set MKL_NUM_THREADS=8
  matlab
  ```

  `OMP_NUM_THREADS` limits the compiled assembly (and PARDISO, if `MKL_NUM_THREADS` is not set); `MKL_NUM_THREADS` limits PARDISO. `maxNumCompThreads` in MATLAB affects neither.

- **Shared computers.** Set both to the number of cores you are allowed to use.

- **Several Úa runs at the same time** (separate MATLAB processes, or `batch` jobs): each run would start one thread per logical processor, and they would compete. Give each run its share, for example 4 runs on 16 cores: `OMP_NUM_THREADS=4` (and `MKL_NUM_THREADS=4`) in each.

- **Running Úa inside `parfor` or on the workers of a pool** (eg ensembles) has not been tested with the compiled assembly. If you do, limit the threads per worker in the same way, so that workers times threads does not exceed the number of cores.

9 Linux

On Linux you build the MEX files once yourself; after that, everything works as on Windows. The included MEX files are for Windows only (`.mexw64`); on Linux MATLAB looks for `.mexa64` files, so until they are built, Úa uses the vectorised assembly and says that the MEX files were not found.

The compiled assembly has so far only been tested on Windows. The steps below are those expected for Linux; check the result with the regression test.

Requirements

- MATLAB Coder.
- A GCC version supported by your MATLAB release (see MathWorks' list of supported compilers). GCC includes OpenMP.

Steps

1. Check that the C compiler is selected (usually `gcc` already is):

   ```
   mex -setup C
   ```

2. Build in a fresh session:

   ```
   matlab -batch "addpath(genpath('/path/to/UaSource')); BuildUaCompiled"
   ```

   This writes the 13 MEX files as `.mexa64` and the build information `UaCompiledBuildInfo_mexa64.mat` to `UaSource/UaCompiled`. For compilers it does not recognise, `BuildUaCompiled` builds `SparseFromCSC` without OpenMP and says so; it then still works, only copies serially.

3. Run the regression test (Section 11); all 88 tests should pass:

   ```
   results=runtests("TestUaCompiled"); table(results)
   ```

4. Switch the compiled assembly on as on Windows (Section 2, step 2).

Clusters

- Build once per MATLAB release, on a node with the same operating system and processor type as the compute nodes; the `.mexa64` files can then be used on all of them through a shared `UaSource`.
- Do not rebuild while runs are using the MEX files.
- Give each job its cores through `OMP_NUM_THREADS`, for example with SLURM:

  ```
  #SBATCH --cpus-per-task=16
  export OMP_NUM_THREADS=$SLURM_CPUS_PER_TASK
  matlab -batch "run('MyUaRun.m')"
  ```

- The `.mexa64` files and `UaCompiledBuildInfo_mexa64.mat` are not in the git repository. Keep them out of commits from Linux machines (or add `*.mexa64` and `UaCompiled/UaCompiledBuildInfo_mexa64.mat` to `.gitignore`).

macOS is not covered: Apple's compiler has no OpenMP, and the build has not been tried there.

10 Troubleshooting

Most problems come from MEX files that are missing, outdated or built for another MATLAB release. Úa detects these and uses the vectorised assembly, with a message saying why; rebuilding the MEX files removes the cause. Switching the options off (`CompiledKernel=false`) always gets a run going again.

Symptom	Cause	Remedy
Message *... MatrixAssembly: compiled kernel not used, as ...*	An option, element type or file that the compiled assembly cannot handle; the message says which.	See Section 4. The run continues with the vectorised assembly.
No speed-up, and no message	The switch is off, or the spmd option of that assembly is on: the vectorised assembly runs on the spmd workers.	Check the three switches, and set the spmd options to `false`.
Message *compiled kernel not used, as the MEX files are outdated: ...* or *... built with another MATLAB release ...*	A source file of the MEX files changed since they were built (eg after `git pull`), or another MATLAB release.	Rebuild them (Section 6). Until then, the vectorised assembly is used.
Message *the compiled kernel failed (... Invalid MEX-file ...)*, or an error about the class, size or fields of an input	MEX files built for another operating system, damaged, or older than their sources.	Rebuild them. The vectorised assembly is used for the rest of the session.
Built the MEX files, but Úa still says they were not found, outdated or failed	Úa checks once per session whether the MEX files exist and are up to date, and remembers the answer (and a failure).	`clear CompiledAssemblyCheck`, or restart MATLAB.
`BuildUaCompiled`: *Unable to copy file ...* for a MEX file	The MEX file is loaded in this or another MATLAB session (MEX files with OpenMP are not released by `clear mex`).	Close the other sessions, restart MATLAB and build first, or use `matlab -batch` (Section 6).
`BuildUaCompiled`: *No C compiler selected*	No compiler, or none selected.	Install one (Windows: MinGW-w64 add-on), then `mex -setup C`.
`BuildUaCompiled` reports an error in a UaMain function (eg after editing `BasalDrag.m`)	The change uses something MATLAB Coder does not support.	See the rules in *Úa: Using the compiled assembly – Notes for Developers*; the error report names the line.

Symptom	Cause	Remedy
Error *Complex-valued input to the uv assembly* (or h assembly)	A complex value in the input fields, which the compiled kernels do not accept.	Find where the complex value comes from; it is not expected in a normal run.
Warning *MUA has no h assembly pattern ...*	The mesh was not created or updated by `CreateMUA.m` or `UpdateMUA.m` (eg a mesh from your own code).	Call `UpdateMUA.m` on it; otherwise the pattern is recreated in every call, which is slow.
Error about `CtrlVar.MassBalance.Evaluation` (*... no longer available*)	Evaluating the mass balance at integration points has been removed.	Remove this setting; the mass balance is always evaluated at the nodes.
The computer is slow for other work during a run, or parallel runs are slower than expected	The compiled assembly and PARDISO use all processors.	Limit them with `OMP_NUM_THREADS` and `MKL_NUM_THREADS` (Section 8.4).
Results differ by more than rounding errors from the vectorised assembly	Not expected.	Run the regression test (Section 11); if it passes but your results differ, report it with the `CtrlVar` settings.

11 Regression test

A regression test checks that the compiled assemblies give the same results as the vectorised ones, for all supported options. Run it after building the MEX files, after updating Úa, or if in doubt:

```
results=runtests("TestUaCompiled");    % 88 tests, about 30 s
table(results)
results=runtests("TestUaCompiled","ProcedureName","testH*");   % only the h tests (
    testUv*: uv, testUvh*: uvh)
```

All tests should pass. The test (`UaSource/UaCompiled/Tests/TestUaCompiled.m`) needs no input files: it builds a small synthetic ice sheet itself (grounded and floating ice, a calving front, thin ice near `ThickMin`), and compares the residuals and matrices of the compiled and the vectorised assemblies for all sliding laws, SUPG tau options, thickness penalty functions, element types and quadrature rules, and other options. It also solves one uv system, one uvh time step and one h system with both, and checks that the MEX files are up to date. For each case it first checks that the compiled path is actually used. A failing test names the case and the quantity, eg *h, option Base: f0*, with the difference found.

If tests fail because the MEX files are missing or outdated, build them first (Section 6). If they fail with up-to-date MEX files, please report it.

12 Performance

Measured on a 16-core Windows desktop, MATLAB R2026b, with the default number of threads; your figures will differ with the machine.

A single assembly (vectorised: serial, unless stated):

Assembly, mesh	Vectorised, full / residual only	Compiled, full / residual only	Speed-up
uv, Greenland nx200 (197,000 elements)	1,494 / 806 ms	100 / 38 ms	15× / 21×
uvh, same mesh	3,903 / 1,433 ms; with spmd (4 workers) 1,681 / 477 ms	186 / 58 ms	21× / 25×
h, Greenland nx150 (111,000 elements)	237 / 141 ms	20 / 7 ms	12× / 20×

Whole runs (profiled; "vectorised" as previously recommended, with spmd for the uv and uvh assembly where stated):

Run	Vectorised	Compiled	Speed-up
Greenland, uv-h scheme, nx150, 20 time steps	314 s (uv with spmd)	108 s (uv and h compiled)	2.9×
Greenland inversion (A, B, C), nx100, 2 iterations	45 s (uv with spmd)	29 s (uv compiled)	1.5×
Greenland, implicit uvh, nx150, 1 year	779 s (spmd, MATLAB solver)	189 s (uvh compiled, PARDISO)	4.1×
Thule, implicit uvh, 0.1 yr, mesh adaptation and level set	209 s (serial)	146 s (uvh compiled)	1.4×

The gain grows with the mesh size and the number of cores. Once the assembly is compiled, other parts take most of the remaining time: in the uv-h run, the linear solves (26 s), the compiled kernels (about 30 s together) and setting up and writing the run (about 17 s); in the inversion, writing the adjoint restart file (10 s).